Sistemas Operativos II

Práctica 4 - Sincronización de procesos con paso de mensajes

Objetivo

Conocer el funcionamiento básico de pase de mensajes como mecanismo de solución al problema de las carreras críticas y desarrollar habilidades en su manejo.

1. Paso de mensajes

1. Repasar la solución vista en clase al problema del productor-consumidor con pase de mensajes.
2. Programar la solución al problema del productor-consumidor utilizando pase de mensajes entre dos procesos según la plantea Tanenbaum. El código del productor debe llamarse *prod.c* y el del consumidor *cons.c*. Ten en cuenta lo siguiente:
   - El tamaño de cada buffer debe ser N=5.
   - El productor debe llamar a una función para producir items (su tarea principal). Esta tarea consistirá en leer los carácteres de un fichero de texto plano, que será el primer argumento de la línea comando al ejecutar el proceso. Deben leerse todos los carácteres del fichero, que constituirán los items enviados al cosumidor. La lectura de cada carácter debe ir acompañada por una espera de un número aleatorio de entre 0 y T microsegundos usando la función *usleep*. El valor de T debe ser el segundo de los argumentos de la línea comando al ejecutar el proceso.
   - El consumidor debe llamar a una función para consumir items (su tarea principal). Esta tarea consistirá en almacenar en un fichero, cuyo nombre será el primer argumento de la línea comando al ejecutar el proceso, todos los carácteres recibidos. La lectura de cada carácter debe ir acompañada por una espera de un número aleatorio de entre 0 y T microsegundos usando la función *usleep*. El valor de T debe ser el segundo de los argumentos de la línea comando al ejecutar el proceso.
   - El buffer de items debe funcionar como una cola FIFO (First In First Out), de manera que el primer item enviado sea el primero consumido.
   - Ambos procesos deben ejecutarse en terminales diferentes.
   - Al final del enunciado hay una plantilla para el código del productor. Ten en cuenta que:
     - Falta la función productor(), en la que se deben usar las funciones *mq send* y *mq receive* para enviar y recibir mensajes.
     - Completa el código del productor, y escribe el del consumidor.
     - Para compilar debes usar la librería -lrt (Realtime Extensions library).

```c
#define MAX_BUFFER 5          /* tamaño del buffer */

/* cola de entrada de mensajes para el productor */
mqd_t almacen1;
/* cola de entrada de mensajes para el consumidor */
mqd_t almacen2;

void main(void) {
    struct mq_attr attr;        /* Atributos de la cola */

    attr.mq_maxmsg = MAX_BUFFER;
    attr.mq_msgsize = sizeof (char);

    /* Borrado de los buffers de entrada
       por si existían de una ejecución previa*/
    mq_unlink("/ALMACEN1");
    mq_unlink("/ALMACEN2");

    /* Apertura de los buffers */
    almacen1 = mq_open("/ALMACEN1", O_CREAT|O_WRONLY, 0777, &attr);
    almacen2 = mq_open("/ALMACEN2", O_CREAT|O_RDONLY, 0777, &attr);
    if ((almacen1 == -1) || (almacen2 == -1)) {
        perror ("mq_open");
        exit(EXIT_FAILURE);
    }

    productor();

    mq_close(almacen1);
    mq_close(almacen2);

    exit(EXIT_SUCCESS);
}

productor(void) {
    /* ToDo */
}
```

2. FreeRTOS

1. Funcionalidade básica de FreeRTOS. Proba os seguintes códigos, dispoñibles no Campus Virtual, e fai un informe explicando o seu funcionamento e comportamento:

   a) Compila e executa o programa "simple_task"(make rtos_simple_task, ./rtos_simple_task). Que fai? Comenta nun informe arquivo simple_task.c, indicando que realizan as funcións e que é cada parámetro. Como se pasan parámetros as funcións de task?

   b) Compila e executa o programa "simple_task_stop_resume"(make rtos_simple_task_stop_resume, ./rtos_simple_task_stop_resume). Que fai? Comenta no informe o arquivo simple_task_stop_resume.c, indicando que realizan as funcións e que é cada parámetro (so as novas respecto ao anterior). Como obten vStopResume o handle do outro task? Porqué o pode facer?

   c) Compila e executa o programa "simple_task_priorities"(make rtos_simple_task_priorities, ./rtos_simple_task_priorities) (No PC da aula igual é mellor quitar un 0 a DELAY_LOOP_COUNT). Que fai? Comenta no informe o arquivo simple_task_priorities.c, indicando que realizan as funcións e que é cada parámetro (so as novas respecto ao anterior). Como realizan a espera as tarefas? Que pasa se cambias a prioridade da tarefa "S2.ª PRIORITY2? Porque?

d) Compila e executa o programa "simple_task_until"(make rtos_simple_task_until, ./rtos_simple_task_until). Que fai? Comenta no informe o arquivo simple_task_until.c, indicando que realizan as funcións e que é cada parámetro (so as novas respecto ao anterior). Que diferenza hai entre vTaskDelay() e vTaskDelayUntil()?

e) Compila e executa o programa "task_cont_vs_periodic"(make rtos_task_cont_vs_periodic, ./rtos_task_cont_vs_periodic). Teras que paralo con CRTL-C pronto e ver a terminal cara atrás, saca moita saida. Que fai? Comenta no informe o arquivo task_cont_vs_periodic.c, indicando que realizan as funcións e que é cada parámetro (so as novas respecto ao anterior). Como explicas o comportamento?

f) Compila e executa o programa "task_idle_hook"(make rtos_simple_idle_hook, ./rtos_task_idle_hook). Que fai? Comenta no informe o arquivo task_idle_hook.c, indicando que realizan as funcións e que é cada parámetro (so as novas respecto ao anterior). Porque funciona se nunca se inicia unha tarefa con vApplicationIdleHook()?

2. Funcionalizade avanzada de FreeRTOS:

   a) Compila e executa o programa "task_semaforos"(make rtos_task_semaforos, ./rtos_task_semaforos). Que fai? Como funciona? Proba a incluir vTaskDelay() e modificar o número de tarefas para comprobar o seu funcionamento. Fíxate nos comentarios //COMENTAR no código para incluir os teus propios comentarios no informe. Que ocorre cando cambias a prioridade das tarefas çonsumidor.ª PRIORITY2, cambia o comportamento? Porque?

   b) Compila e executa o programa rtos_blinky"(make rtos_main_blinky, ./rtos_blinky), o programa oficial de demo de FreeRTOS. Que fai? Que fai cada task? Como se comunican os tasks? (o código main_blinky.c está no cartafol FreeRTOS)

3. Exercicios opcionais

1. Intenta replicar os exemplos dos algoritmos RMS e EDF vistos en clase (tomados do Tanenbaum) usando FreeRTOS:

   - Podes ter unha tarefa que imprima por pantalla o tempo límite para o remate de cada proceso.
   - Podes simular cada proceso cunha tarefa que execute operacións inútiles (como un bucle de operacións matemáticas) para ter un tempo de execución aproximado. Inicialmente usa un tempo de execución baixo para que o factor de utilización sexa baixo e sexa planificable. Estas tarefas imprimirán por pantalla unha mensaxe dicindo que rematou e o seu tempo de execución.
   - Para que os procesos se inicien cando deben podes ter delays dentro deles mesmos ou usar a tarefa que leva a conta do tempo para reinicialos.
   - Axusta as prioridades de maneira adecuada
   - Podes axustar o tempo no que se teñen que executar as tarefas para que sexa máis sinxelo de simular (en vez de realizar as tarefas 30, 20 ou 25 veces por segundo facelo cada 10 segundos, por exemplo).
   - Varía o tempo de execución das tarefas ata que o programa non sexa planificable. Intenta calcular o factor de utilización a partir de onde non é planificable.

4. Entrega

- Sobre el *Paso de Mensajes*, punto 1:
  - Deben subirse los códigos en C con comentarios exhaustivos sobre su funcionamiento y manejo.
  - Un informe corto sobre el comportamiento del programa, 2 páginas como máximo.
  - La fecha de entrega es la indicada en el Campus Virtual.
- Sobre FreeRTOS:
  - Un informe sobre o comportamento dos códigos, contestando as preguntas feitas no punto 2.
- Sobre os opcionais:
  - Os códigos comentados.
  - Un informe sobre o funcionamento do código e o seu comportamento.